EDITOR: Amit P. Sheth, amit@sc.edu

DEPARTMENT: KNOWLEDGE GRAPH

Symbolic-in-the-Loop: Leveraging Knowledge Graph Engineering to Ensure High-Assurance Generative Artificial Intelligence Outcomes

Venkatesan Nadimuthu, *University of South Carolina, Columbia, SC, 29201, USA*

Archy Das, *Sikkim Manipal Institute of Technology, Majitar, East Sikkim, 737136, India*

Vatsalya Soni, *Delhi Technological University, New Delhi, Delhi, 110085, India*

Amit P. Sheth, *University of South Carolina, Columbia, SC, 29201, USA; and Indian AI Research Organization, GIFT City, Gujarat, 382355, India*

Large language models excel at complex reasoning and natural language processing. However, probabilistic next-token prediction makes them inherently vulnerable to factual hallucinations. This unpredictability prevents their unmonitored use in highly regulated, mission-critical enterprise environments. To support high-assurance autonomous execution, we propose shifting from context engineering to knowledge graph (KG) engineering. We introduce the symbolic-in-the-loop (SITL) verification framework, a novel neurosymbolic architecture. Traditional retrieval augmented generation uses KGs for pregeneration context. Instead, SITL repositions the KG at the end of the pipeline as a formal verification layer. This postgeneration symbolic check reliably grounds probabilistic neural output into a deterministic ontology. The architecture seamlessly accommodates dynamic KG updates. This ensures validation against the most current symbolic state of an enterprise. We evaluate SITL across applications ranging from personal automation to multiagent orchestration. Experiments show this terminal KG approach successfully eliminates semantic drift. Ultimately, SITL ensures provably accurate outputs, replacing traditional human-in-the-loop models with scalable automation.

Artificial intelligence (AI) has achieved remarkable performance through deep learning architectures over the past decade. This success has catalyzed the widespread adoption of agentic AI within enterprise environments. Applications are evolving from simple single-agent patterns to sophisticated, autonomous multiagent systems. Despite these advancements, the operational efficacy of autonomous systems remains bounded by a severe vulnerability. Industry practitioners widely refer to this fundamental limitation as the enterprise "trust gap."

Digital Object Identifier 10.1109/MIC.2026.3680545
Date of current version 15 May 2026.

INTRODUCTION

Core Problem: Probabilistic Vulnerability

Autonomous agents are currently tied to the probabilistic mechanisms of large language models (LLMs). These underlying models operate on statistical pattern matching rather than formal logical deduction: 1) *inherent stochasticity*, as relying on statistical next-token prediction across vast latent spaces makes generated

outputs inherently nondeterministic; 2) *imperative hallucination*, where the generation of linguistically plausible but factually incorrect claims remains a fundamentally unresolved challenge; and 3) *enterprise risk*, because unpredictable hallucinations create unacceptable legal and operational hazards, such as fabricated precedents or incorrect dosages that current models cannot reliably prevent.[1]

The Human-in-the-Loop Bottleneck: A Barrier to Scale

LLMs fundamentally operate as opaque "black boxes" during text generation. Therefore, enterprises remain highly skeptical of delegating critical tasks to unmonitored AI. To mitigate ungrounded outputs, governance teams uniformly mandate human-in-the-loop (HITL) validation protocols. These protocols must be satisfied before any production deployment.

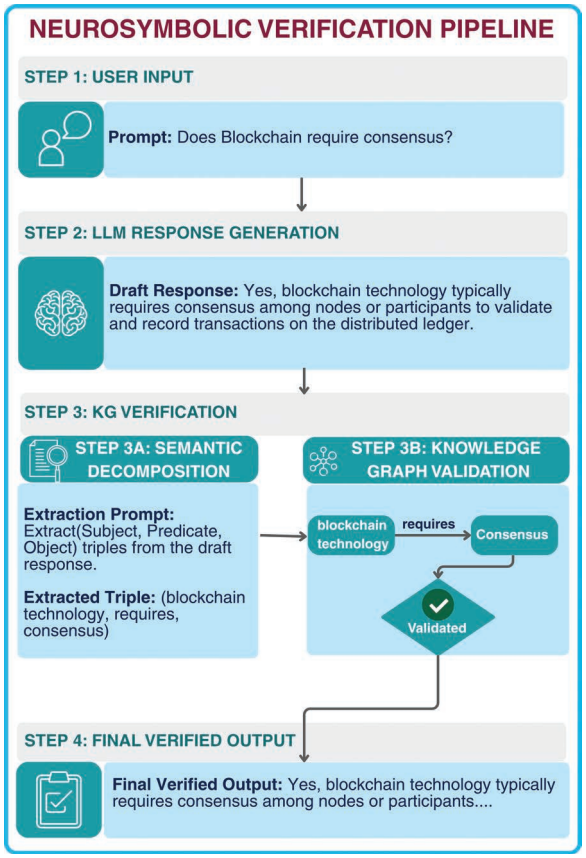

FIGURE 1. Neurosymbolic verification pipeline. Step 1: User input, Step 2: LLM response generation, Step 3: KG verification via semantic decomposition and triples' validation (e.g., [blockchain technology] requires [Consensus]), and Step 4: final verified output.

While HITL acts as a necessary safety net, it introduces critical limitations: 1) *Latency*, where human review interrupts the microsecond speed of AI execution, reverting process times back to traditional human scales; 2) *cost and cognitive load*, as relegating subject matter experts to routine verification tasks imposes significant overhead and diminishes the core efficiency gains expected from autonomous systems; and 3) *scalability failure*, because mandating manual authorization for every output fundamentally bottlenecks the throughput of autonomous agentic systems.

Epochs of Generative AI Optimization

To contextualize our structural intervention, we examine the trajectory of generative AI optimization. The evolution of AI engineering can be categorized into three distinct epochs:

- *Prompt engineering*, which relied on heuristic manipulation of the input prompt. However, this method proved fragile and highly sensitive to model updates.
- *Context engineering*, where retrieval augmented generation (RAG) introduced nonparametric augmentation via vector embeddings[2] to provide external grounding prior to text generation. Although RAG provides relevant context, it cannot deterministically restrict the model from ignoring or misinterpreting the retrieved documents during generation.
- *Knowledge graph (KG) engineering*, which formalizes domain logic into a verifiable symbolic substrate to ensure provable correctness by merging stochastic generation with deterministic logic.

Proposed Solution: Symbolic in the Loop

Building on this third epoch, we propose a novel neurosymbolic paradigm (see Figure 1). This approach effectively replaces the HITL requirement with a symbolic-in-the-loop (SITL) architecture, characterized by three core pillars:

- *Terminal architecture*, which positions a deterministic KG at the end of the neural generation pipeline.
- *Absolute adjudicator*, serving as the final symbolic gatekeeper before executing any system commands.[3,4]

- *High-assurance compliance*, which automatically validates or rejects stochastic neural output against the rigid guardrails of a curated enterprise KG.[5]

RELATED WORK: BRIDGING NEURAL AND SYMBOLIC AI

Recent advances in LLMs have significantly improved natural language understanding. However, addressing hallucination, lack of factual grounding, and difficulty handling structured knowledge requires bridging neural and symbolic architectures. The existing literature can be categorized into four major research directions.

KG–LLM Integration Frameworks

Several studies propose frameworks that tightly integrate KGs with LLMs to improve the reliability of reasoning. These approaches generally fall into three categories: 1) *complementary strengths*, where LLMs provide flexible reasoning while KGs offer structured factual grounding; 2) *neurosymbolic orchestration*, such as the DYNO framework,[6] which integrates neural task planning with symbolic workflow validation to improve reliability in agentic AI; and 3) *colearning paradigms*, exemplified by structure-oriented RAG, where LLMs assist in KG construction while the KG simultaneously guides LLM reasoning.[3]

KG Construction and Representation Learning

Traditional KG construction relies on costly manual annotation. Recent studies propose automated construction methods using LLMs. Several studies have proposed specialized frameworks to bridge the gap between neural generation and symbolic grounding. These include:

- *LightKGG*, which demonstrates that small LLMs can efficiently extract entities and relations with low computational requirements.[7]
- *KGFIT*, which integrates open-world knowledge into KG embeddings through hierarchical entity structures.[8]
- *Conceptformer*, proposing a vector-based knowledge injection approach.[9]
- *EMPWR Platform*, a KG lifecycle management framework for automated data ingestion, ontology/KG construction, and semantic enrichment.[10]
- *Multimodal KGs*, which uses automated construction pipelines that merges bottom-up extraction with top-down schema refinement. This enables scalable integration of heterogeneous enterprise data like text and images.[11]

Retrieval Augmented Reasoning and Multihop Inference

Recent frameworks introduce advanced mechanisms to enforce structured neural reasoning, including:

- *T-RAG (triplet-driven thinking)*, which decomposes complex queries into atomic triplets and iteratively resolves them using a structured knowledge base.[12]
- *Graph constrained reasoning*, which introduces a decoding constraint mechanism, utilizes a KG-Trie index to ensure that generated reasoning paths correspond to valid graph relations.[4]
- *Dynamic knowledge updating*, exploring paradigms where language models interact seamlessly with continuously evolving enterprise KGs.[13]

Evaluation and Benchmarking

Recent advances in evaluation methodologies include: *KG-LLM-Bench*, which introduces a dedicated benchmark to evaluate graph textualization strategies,[14] and *reasoning transparency*, encompassing frameworks that measure the correctness of reasoning paths rather than merely the accuracy of the final answer.[15]

The Gap

Despite significant progress, issues related to resolving conflicts between parametric LLM knowledge and external KGs remain. Crucially, most of the existing literature utilizes KGs to *enhance* inputs (pre-generation). Our work fills a critical gap by utilizing KGs to *restrict and verify* outputs (post-generation).

ARCHITECTURE FOUNDATIONS

Operationalizing the SITL paradigm requires a robust structural foundation. This foundation bridges stochastic neural networks with the deterministic rigor of formal logic.

Ontology and KG Topologies

Our symbolic verification layer strictly separates the conceptual schema from instantiated data. By enforcing a closed-world assumption within the verification boundary, unverified statements are strictly evaluated as false, ensuring mandatory enterprise safety. We

formalize this using standard description logic semantics. The enterprise domain is defined as a tuple $\mathcal{D} = \langle \mathcal{T}, \mathcal{A} \rangle$:

- *Ontology ($\mathcal{T}$/T-Box):* Defines the ontological schema, semantic constraints, and logical invariants (e.g., enforcing a 15% upper bound on promotional discounts).
- *Knowledge graph ($\mathcal{A}$/A-Box):* Comprises real-world entity relationships represented as verified triples $t = (s, p, o) \in \mathcal{A}$.

To pass the execution gate, any stochastically generated neural output O must be logically entailed by the domain: $\mathcal{D} \models O$.

Democratizing Symbolic Construction

Historically, the "knowledge acquisition bottleneck" severely impeded enterprise neurosymbolic AI due to labor-intensive, manual KG construction. Today, automated platforms, such as EMPWR,[10] resolve this limitation through advanced entity-relation extraction paradigms. Automated parsing now enables the seamless instantiation of complex ontologies, successfully eliminating manual resource description framework overhead. This critical democratization facilitates the rapid deployment of the dynamic verification gateway essential for SITL implementation.

PROPOSED NEUROSYMBOLIC QUALITY CONTROL FRAMEWORK

Building upon these topological foundations, we introduce a comprehensive quality control (QC) framework that initiates a continuous, four-stage verification cycle within the execution pipeline: 1) *neural inference*, where the generative LLM processes the initial prompt to draft a probabilistic, unverified candidate response; 2) *semantic decomposition*, where a secondary extraction model rigorously translates the unstructured natural language into standardized atomic triples; 3) *symbolic validation*, which queries these triples directly against the authoritative KG to verify assertions against the deterministic guardrails embedded in D; and 4) *iterative refinement*, where detected logical violations trigger a deterministic error signal, forcing iterative re-generation until the output strictly aligns with the KG.

Understanding the Loop Mechanics

Algorithm 1 demonstrates the operational core of the SITL architecture. The MAX_ITER = 5 threshold functions as a critical escape hatch. Bounding k prevents infinite loops caused by persistent hallucination attempts. The system explicitly trades generative persistence for absolute factual safety. Execution aborts deterministically if graph constraints remain unmet after five attempts. This ensures that graph sparsity results in a safe failure rather than a forced hallucination.

Algorithm 1. SITL Verification with Syntactic and Semantic Guardrails.

Require: User prompt P, Enterprise domain $\mathcal{D} = \langle \mathcal{T}, \mathcal{A} \rangle$
Ensure: Verified response R or Abort

```
k ← 0, MAX_ITER ← 5
R ← LLM_Generate(P)
while k < MAX_ITER do
    O ← Semantic_Parse(R)
    if O = Invalid then
        E ← "Format Violation:Response Parse Error"
    else
        V ← (D ⊨ O)
        if V = True then
            return R        ▷ Rules verified successfully
        end if
        E ← Semantic_Violations(O, D)
    end if
    R ← LLM_Regenerate(P, E)
    k ← k + 1
end while
return Abort: Escape hatch triggered
```

MULTI-TIERED APPLICATION SPECTRUM

We categorize SITL's application across four hierarchical tiers to evaluate its scalability. This spectrum demonstrates architectural evolution from personal automation to multiagent ecosystems. Each tier delivers highly specific business outcomes. To evaluate the scalability and operational versatility of the proposed neurosymbolic framework, we categorize its application into four distinct hierarchical tiers, as illustrated in Figure 2. Each tier demonstrates varying degrees of architectural complexity, ranging from isolated personal automation to a deeply integrated, multiagent enterprise ecosystem (see Table 1).

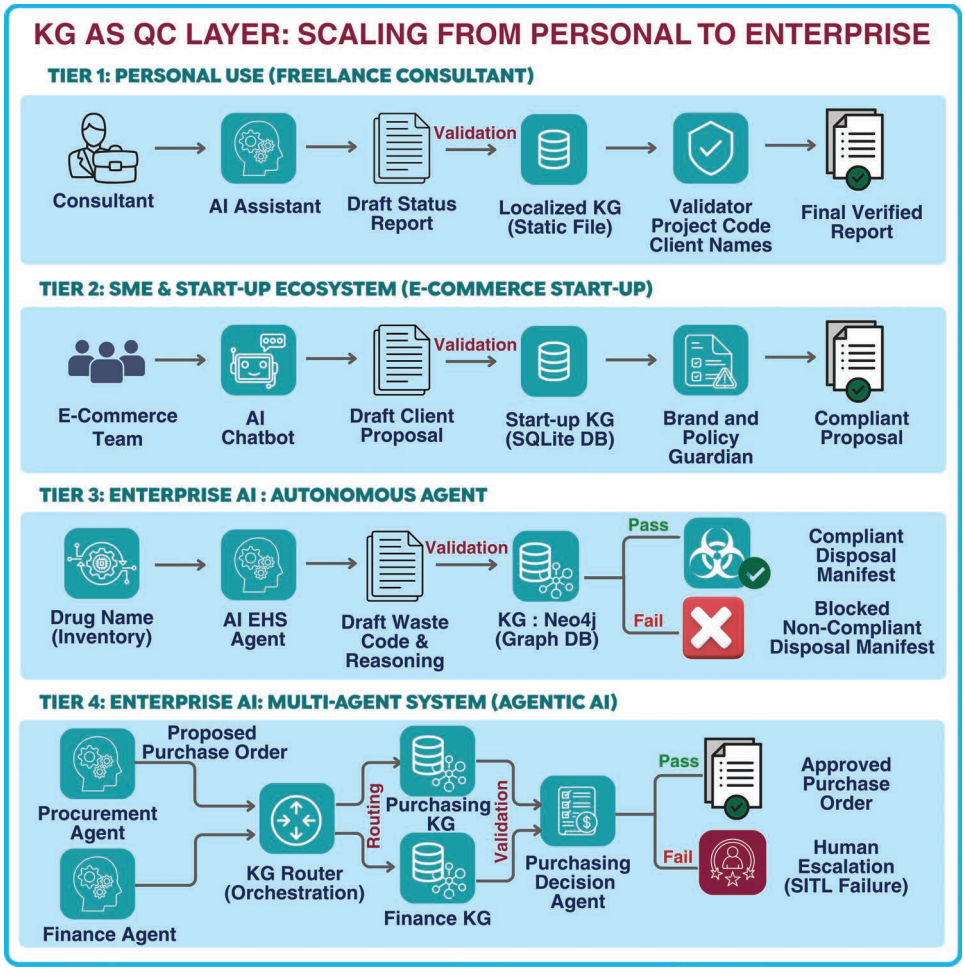

FIGURE 2. Use cases: KG as a QC layer: from Tier 1 (personal localized files) and Tier 2 (small and medium enterprise policy enforcement) to Tier 3 (enterprise autonomous guardrails) and Tier 4 (enterprise multiagent orchestration via KG router). Note: The architecture is illustrated for a direct pass/fail scenario (k = 0).

TABLE 1. Summary of SITL-based KG architectures across four deployment tiers, categorized by architecture type, core validation function, and representative application domains.

Tier and Environment	KG Architecture	Core Function	Example Domain
Tier 1 Personal use (repetitive tasks)	Localized KG (Static JSON)	Personal output validation; hallucination prevention.	Freelance consulting (reporting)
Tier 2 Small and medium enterprises	Unified KG (lightweight database)	Cross-functional; Brand and policy guardian	E-commerce (sales and support)
Tier 3 Enterprise single-agent	Terminal KG (strict T-Box guardrails)	Autonomous regulatory guardrail; replaces HITL.	Manufacturing/Healthcare (safety)
Tier 4 Enterprise multiagent	Distributed KGs via KG router	Complex orchestration; cross-agent collaboration.	Supply chain/procurement

Tier 1: Personal Use for Repetitive Tasks

This foundational tier optimizes individual efficiency for routine knowledge work.

- *Mechanism:* A localized KG validates the generated outcomes. Users build custom KGs via platforms like EMPWR.[10] These are exported as static JSON files for direct AI integration.
- *Use case:* A consultant generates weekly status reports. The KG ensures that the text only references valid project codes.
- *Business outcome:* Eliminates reputational damage from submitting reports with fabricated data points.

Tier 2: Small and Medium Enterprise and Startup Ecosystems

Designed for resource-constrained environments that lack strict departmental boundaries.

- *Mechanism:* Organizations deploy lightweight graph databases for a unified KG. This abstracts domain constraints into an automated QC layer. It acts as a brand and policy guardian.
- *Use case:* An e-commerce startup uses AI to draft client proposals. The unified KG strictly enforces maximum discount limits of 15%.
- *Business outcome:* Accelerates onboarding and democratizes knowledge work. New hires produce compliant assets immediately without the senior HITL review.

Tier 3: Enterprise AI Single-Agent Use Case

Autonomous single-agent systems introduce unacceptable liabilities in specialized, high-risk domains.

- *Mechanism:* The framework establishes the KG as an uncompromising regulatory guardrail. Autonomous agents generate outputs that must comply entirely with the prevailing T-Box.
- *Use case:* An AI environmental health and safety (EHS) agent classifies hazardous materials. The terminal KG validates AI-generated waste codes against regulatory guidelines. It blocks noncompliant exemptions and enforces strict hazardous classifications.
- *Business outcome:* Achieves absolute regulatory compliance. Physically preventing noncompliant actions neutralizes catastrophic enterprise risks.

Tier 4: Enterprise AI Multiagent Use Case

Multiagent AI architectures handle advanced business processes where a monolithic KG is unmanageable.

- *Mechanism:* Organizations manage multiple domain-specific KGs via a KG router. This orchestration layer dynamically routes validation requests to task-specific KGs. It ensures cross-agent state changes remain logically consistent.
- *Use case:* A procurement agent generates a purchase order. The KG router routes requests to both the purchasing and finance KGs. The finance KG executes the final validation for the purchase decision. It blocks state changes if orders exceed allocated funds.
- *Business outcome:* Cross-departmental synchronization prevents multiagent cascade failures, ensuring global enterprise stability.

PROTOTYPE AND BUILD

We built an experimental SITL prototype that rigorously eliminates hallucinations through deterministic grounding.

Multiagent Verification Pipeline Mechanics

The prototype operates as a multiagent neurosymbolic AI, orchestrating the execution pipeline via three specific operational subagents: 1) the *generative agent*, where an initial LLM drafts a probabilistic natural-language response based on the user query; 2) the *extractive agent*, which utilizes a secondary LLM to decompose the draft into core factual claims isolated as subject–predicate–object triples; and 3) the *symbolic engine*, which programmatically queries the backing graph database to deterministically verify these extracted relationships.

Edge Case Resolution and Qualitative Analysis

We stress-tested these mitigation strategies against technical edge cases using 100 diverse benchmark queries. These covered factual, adversarial, verbose, and out-of-domain inputs. The system manages these anomalies through three mechanisms: 1) *out-of-domain interception*, where a semantic router immediately blocks unrelated input to preserve application programming interface (API) bandwidth; 2) *adversarial short-circuiting*, which safely returns an unverified status when the generative agent lacks sufficient evidence; and

3) *verbose distillation*, where the system rejects unverified conversational elements and forces the LLM to rewrite outputs strictly using validated triples.

IMPLEMENTATION: NEUROSYMBOLIC PIPELINE

Operationalizing this framework requires a definitive verification pipeline. It sits strategically between the LLM's generative output and final system actions.

System Architecture

The practical implementation comprises five interoperating layers:

- *Layer 1: Enterprise data ingestion*, which aggregates foundational business knowledge from diverse sources such as corporate policies, customer relationship management records, and standard operating procedures.
- *Layer 2: Ontology construction and KG instantiation*, where this knowledge is formalized and instantiated into a live, queryable graph reflecting current enterprise states.
- *Layer 3: Neural generation layer*, in which the LLM processes a prompt to produce a candidate output.
- *Layer 4: Symbolic verification layer*, which rigorously decomposes the candidate output and restricts the LLM strictly to deterministic JSON triple extraction to prevent downstream hallucinations.
- *Layer 5: Execution gate layer*, acting as the final software boundary that forwards verified outputs to the API execution layer while logging failed outputs as audit objects for human escalation.

Knowledge Acquisition and Automated Graph Construction

To ensure that the graph reflects real-time enterprise states, we leverage automated extraction pipelines that execute across four distinct stages: 1) *entity extraction*, which parses multimodal enterprise corpora using advanced natural language processing and named entity recognition to identify core business entities; 2) *relation extraction*, which extracts semantic links connecting these entities through a zero-shot relation extraction workflow; 3) *ontological canonicalization*, which resolves semantic synonymy and removes duplication to ensure that extracted predicates correspond to valid relations; and 4) *continuous update and governance*, incorporating graph versioning and confidence scoring where newly extracted facts await HITL approval before live commitment, thus preserving governance rigor while automating downstream generative execution.

Verification Engine and Failure Typology

During semantic verification ($\mathcal{D} \models O$), the semantic triple parser evaluates candidate graph fragments against a nuanced failure typology to dictate the appropriate error signal: 1) *factual failures*, where out-of-graph entities are conservatively rejected to prevent unverified claims; 2) *policy failures*, triggered when the output explicitly violates an encoded enterprise business rule; 3) *schema failures*, occurring if relations are structurally inconsistent with the defined ontology; 4) *state failures*, where the proposed actions are logically invalid within the current workflow state; and 5) *safety failures*, activated when outputs exceed predefined operational, physical or regulatory limits.

QUANTITATIVE EVALUATION

We evaluated system performance using semantic drift, quantified via the factual error rate over extracted knowledge triples. The baseline corresponds to a raw LLM pipeline without symbolic verification, whereas the SITL pipeline incorporates structured extraction, KG validation, and deterministic fallback mechanisms (Table 2).

Both systems were tested against an identical benchmark of 100 diverse prompts.

To appropriately contextualize the system's output volume, we distinguish between extraction yield (the number of structurally valid claims parsed from the LLM's raw text) and verification precision (the proportion of those claims successfully validated as true against the KG).

TABLE 2. Semantic drift evaluation of baseline versus SITL pipeline

Metric	Baseline	SITL
Total queries	100	100
System abstentions	0	90
Extracted triples	60	10
Triples rejected by KG	51	0
Final verified triples	9	10
Triple-level factual error rate	85%	0.00%
Drift reduction	–	100%

Analysis

In the baseline evaluation, the raw LLM operates as a high-volume, low-accuracy generator (Figure 3).

Although it attempts to answer all 100 queries, the absence of structural enforcement yields verbose, ungrounded, and inconsistently formatted output. Consequently, the extraction phase successfully parses 60 structurally valid triples. Upon automated verification, 51 of these 60 triples (85%) were flagged as hallucinations entirely absent from the enterprise graph, leaving only nine factually correct assertions. This demonstrates that in unconstrained models, the raw output volume does not translate to reliability (Figure 4).

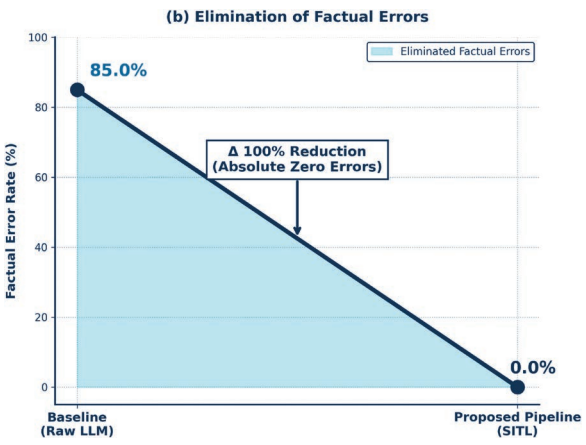

FIGURE 3. Semantic drift reduction showing the decrease in hallucination rate from 85% (baseline) to 0.00% (SITL pipeline).

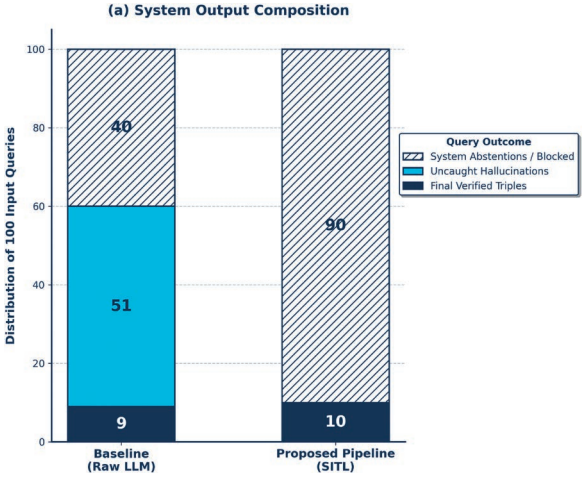

FIGURE 4. A comparative breakdown of query outcomes, demonstrating the SITL pipeline's ability to intercept invalid prompts and output strictly verified triples compared to an unconstrained baseline.

In contrast, the SITL pipeline functions as a deterministic filtration system that strictly prioritizes precision over recall. The apparent reduction in extraction yield from 100 input queries down to 10 final triples is not a performance deficit, but the direct result of enforced epistemic boundaries. The system systematically triggers controlled abstention for out-of-domain and adversarial prompts, safely returning zero triples for invalid queries rather than hallucinating an answer. For the remaining factual queries, the pipeline mandates absolute schema alignment, conservatively rejecting claims that suffer from graph sparsity.

Crucially, through its iterative constraint loop, the SITL pipeline successfully recovered and formatted one valid fact that the baseline dropped, resulting in 10 final verified triples (Figure 4). By intentionally trading unreliable volume for an executable trust boundary, the proposed architecture achieved a 0.00% factual error rate, completely eliminating semantic drift.

DISCUSSIONS, LIMITATIONS, AND FUTURE WORK

Although SITL eliminates semantic drift within its operational boundary, three structural constraints limit its current applicability and motivate future research. The evaluation dataset was intentionally designed to stress test how queries are filtered across different stages of the SITL pipeline before KG validation. Although such distributions may not reflect real-world enterprise scenarios, they help to demonstrate strict verification behavior of the system. For example, in a Tier 3 EHS classification system, the model would reject any hazardous classification not explicitly defined in the KG to ensure regulatory compliance. In practice, companies can adjust this strictness based on domain needs, allowing the architecture to balance precision and coverage without compromising safety.

Precision–Recall Tradeoff

The achievement of a 0.00% factual error rate reflects a deliberate precision–recall tradeoff rather than flawless neural generation. To ensure strict verification precision, the SITL architecture intentionally sacrifices response coverage. In mission-critical enterprise domains, treating a false negative as an acceptable operational cost is necessary to completely eliminate the risk of a false positive. Thus, the 100% reduction in semantic drift is a successful algorithmic constraint, not a generative anomaly. However, this strict verification introduces operational limitations.

Inadequate Knowledge Limitation (Graph Sparsity)

The primary limitation is the absolute reliance on the completeness of the KG. The execution gate strictly prunes unverified claims under a closed-world assumption. A sparse KG inadvertently rejects factually correct but unmapped responses. This strictly bounds the generative model's heuristic utility.

Configurable Verification Strictness (Levels 0 to 5)

Future iterations will introduce a verification temperature scale to mitigate graph sparsity:

- *Level 5 (maximum strictness):* Enforces cryptographically verified determinism. Mandated for high-stakes autonomous execution.
- *Levels 1 to 4 (intermediate):* Enables administrators to dynamically balance factual rigidity with generative flexibility.
- *Level 0 (no verification):* Bypasses the terminal KG entirely for unconstrained creative brainstorming.

Autonomous Knowledge Evolution and Graph Synthesis

Current pipelines treat KGs statically, causing structural knowledge decay. Future systems will transition to a self-augmenting symbolic layer.[16] Extractive LLMs will identify gaps and autonomously propose high-fidelity triples from trusted external contexts.

Advanced KG Embeddings

Future research will explore deep structural KG embeddings to expand semantic validation. This empowers the execution gate to infer implicit relationships. It validates complex logic chains without hard-coding every nuanced connection.

Context Graphs

Modern enterprises demand the transition from static KGs to dynamic context graphs.[17] This captures active metadata, including data lineage and system processes. Integrating this contextual layer enables nuanced adjudications based on real-time compliance.

CONCLUSION

LLMs demonstrate great reasoning ability but suffer from probabilistic factual instability. Our evaluation confirms that unconstrained models produce severe hallucination rates. This highlights critical deployment risks without rigorous, systemic guardrails.

We introduce a neurosymbolic architecture that utilizes a terminal execution gate. It cleanly separates probabilistic language generation from rigid factual verification. The automated SITL protocol successfully replaces the unscalable HITL bottleneck. All accepted outputs strictly satisfy the absolute constraints of enterprise knowledge.

Experiments confirm that this framework eliminates hallucinations entirely. It yields an unprecedented 100% reduction in semantic drift. SITL effectively bridges the AI trust gap in the enterprise. Organizations can safely deploy autonomous generative AI systems at scale. Factual correctness, auditability, and safety remain non-negotiable guarantees.

ACKNOWLEDGMENTS

This work was partially supported by the National Science Foundation under Grant 2335967 "EAGER: Knowledge-Guided Neurosymbolic AI with Guardrails for Safe Virtual Health Assistants." In addition, this work was further refined through collaborative brainstorming sessions and discussions involving members of our research team and collaborators. We acknowledge a robust discussion in our team for enriching earlier drafts.

REFERENCES

1. L. Cai, C. Yu, Y. Kang, Y. Fu, H. Zhang, and Y. Zhao, "Practices, opportunities and challenges in the fusion of knowledge graphs and large language models," *Frontiers Comput. Sci.*, vol. 7, Jul. 2025, Art. no. 1590632, doi: 10.3389/fcomp.2025.1590632.
2. C. Ma, Y. Chen, T. Wu, A. Khan, and H. Wang, "Large language models meet knowledge graphs for question answering: Synthesis and opportunities," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2025, pp. 24,589–24,608, doi: 10.18653/v1/2025.emnlp-main.1249.
3. C. Yang, R. Xu, L. Luo, and S. Pan, "Knowledge graph and large language model co-learning via structure-oriented retrieval augmented generation," *Bull. Tech. Committee Data Eng.*, vol. 48, no. 4, pp. 9–46, 2024.
4. L. Luo, Z. Zhao, G. Haffari, Y.-F. Li, C. Gong, and S. Pan, "Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models," 2024, *arXiv:2410.13080*.
5. P. Jiang et al., "Reasoning-enhanced healthcare predictions with knowledge graph community retrieval," 2024, *arXiv:2410.04585*.

6. R. Garimella, C. Shyalika, R. P. K. Mana, and A. Sheth, "DYNO: Dynamic neurosymbolic orchestrator for multi-agent systems," in *Proc. LLM-Based Multiagent Syst.: Towards Responsible, Reliable, Scalable Agentic Syst.*, 2026.
7. T. Lin, "LightKGG: Simple and efficient knowledge graph generation from textual data," 2025, *arXiv:2510.23341*.
8. P. Jiang, L. Cao, C. D. Xiao, P. Bhatia, J. Sun, and J. Han, "KG-FIT: Knowledge graph fine-tuning upon open-world knowledge," in *Proc. 38th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2024, pp. 136,220–136,258.
9. J. Barmettler, A. Bernstein, and L. Rossetto, "ConceptFormer: Towards efficient use of knowledge-graph embeddings in large language models," 2025, *arXiv:2504.07624.*
10. H. Y. Yip and A. Sheth, "The EMPWR platform: Data and knowledge-driven processes for the knowledge graph lifecycle," *IEEE Internet Comput.*, vol. 28, no. 1, pp. 61–69, Jan./Feb. 2024, doi: 10.1109/MIC.2023.3339858.
11. R. Garimella, H. Y. Yip, R. Venkataramanan, and A. P. Sheth, "Building multimodal knowledge graphs: Automation for enterprise integration," *IEEE Internet Comput.*, vol. 29, no. 3, pp. 76–84, May/Jun. 2025, doi: 10.1109/MIC.2025.3588546.
12. S. Gong, X. Tang, C. Yang, and W. Jin, "Beyond chunks and graphs: Retrieval-augmented generation through triplet-driven thinking," 2025, *arXiv:2508.02435*.
13. D. Jin, J. Yang, X. Wang, J. Zhang, S. Li, and D. He, "A dynamic knowledge update-driven model with large language models for fake news detection," in *Proc. 34th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2025, pp. 3000–3008.
14. E. Markowitz, K. Galiya, G. V. Steeg, and A. Galstyan, "KG-LLM-Bench: A scalable benchmark for evaluating LLM reasoning on textualized knowledge graphs," 2025, *arXiv:2504.07087.*
15. L. Meyer et al., "Evaluating large language models for RDF knowledge graph related tasks - The LLM-KG-bench-framework 3," *Semantic Web*, 2025. [Online]. Available: https://www.semantic-web-journal.net/system/files/swj3869.pdf
16. R. Venkataramanan, C. Shyalika, and A. P. Sheth, "Dynamic multimodal process knowledge graphs: A neurosymbolic framework for compositional reasoning," *IEEE Internet Comput.*, vol. 29, no. 1, pp. 86–92, Jan./Feb. 2025, doi: 10.1109/MIC.2024.3520366.
17. P. Sankar, "Gartner on context graphs: Top insights, capabilities & implementation recommendations for 2026," Atlan, Singapore, Mar. 2026. Accessed: Mar. 18, 2026. [Online]. Available: https://atlan.com/know/gartner-context-graphs/

VENKATESAN NADIMUTHU is a Ph.D. student at the AI Institute of the University of South Carolina, Columbia, SC, 29201, USA. Contact him at: nadimuthuv@sc.edu.

ARCHY DAS is a research intern at the AI Institute of the University of South Carolina, Columbia, SC, 29201, USA, and an undergraduate student at Sikkim Manipal Institute of Technology, Majitar, East Sikkim, 737136, India. Contact her at archy_202200147@smit.smu.edu.in.

VATSALYA SONI is a research intern at the AI Institute of the University of South Carolina, Columbia, SC, 29201, USA and an undergraduate student at the Delhi Technological University, New Delhi, Delhi, 110085, India. Contact him at vatsalyasoni_23cs456@dtu.ac.in.

AMIT P. SHETH is the NCR chair, and a professor at the AI Institute of the University of South Carolina, Columbia, SC, 29201, USA. He is a Fellow of IEEE. Contact him at: amit@sc.edu.